Design and Build Specification

Predictive Multi-Agent System for Airline Disruption Recovery

Agentic AI (26CD7PEAAI), B.E. CSE (Data Science), BMSCE | Version of 26 October 2026

Team: Athish Raj, Dinesh K, Sanchit Rajesh | Instructor: Dr. Kalyan Nagaraj

How to use this document. It is meant to be the single source of truth for the project, for people and for a coding agent. Section 11 gives the build order with acceptance checks. Items marked **UNVERIFIED** were not confirmed against the real source, so check them before relying on them. Do not invent data, results or regulation text. If something is missing, say so and ask. Implementation status: *the team should update this line with what is already built.*

1 Problem, Decision and Scope

When something goes wrong at a hub airport, such as fog closing the runway or an aircraft being grounded, a duty manager has to rework aircraft, crew and passenger plans by hand while the clock is running. One disruption can snowball into missed connections, crew running past legal duty limits, and a large compensation bill. The rules are fixed but the goals pull against each other: cheaper plans tend to strand more passengers.

The one decision supported: *which recovery plan should the duty manager approve?* The system is advisory and human-confirmed. It abstains or escalates when evidence is weak.

Scope limits. One hub airport, one disruption type (fog), a 24-hour window, and roughly 15 aircraft in the synthetic fleet. Real data is used for flights, weather and rules where available. Crew rosters and passenger itineraries are not public, so they are synthetic and anchored to real flights. This hybrid must be stated plainly in any paper.

2 Architecture and Data Flow

The system is a LangGraph graph with a shared, versioned plan state. A Sentinel agent reads weather data and flags a likely disruption, or a real disruption arrives as an event, and either one starts the orchestrator. Soft agents (Rotation, Cost, Passenger) propose changes or score plans. Hard agents (Crew legality, Slot) check plans and can veto them. A Mediator ranks the surviving plans and either recommends one or escalates to the duty manager. Agents never do arithmetic: they call Python tools, query SQLite, and retrieve rule text from a local vector index. Ingestion scripts build the database and index from real sources and generate the synthetic parts.

One round. The Mediator creates a plan skeleton. Rotation and Cost propose changes. Crew and Slot check them and veto illegal ones with a cited reason. Passenger scores the survivors. The Mediator ranks them with the score $\text{score} = w_1\,\text{delay} + w_2\,\text{missed connections} + w_3\,\text{cost} + w_4\,\text{crew buffer penalty}$ (weights configurable), then reflects: does the top plan break any constraint no agent checked? If so it re-plans. It stops at three rounds, at the token budget, or when the improvement is below a threshold.

Escalation rule. Escalate to the duty manager if the top two plans are within a set margin, if any agent abstained, or if retrieval evidence stayed weak after one re-query.

3 Agents

Crew agent retrieval (agentic RAG). Retrieve the clause, grade whether it covers the case (split duty, augmented crew, time-zone crossing), rewrite the query and retrieve again if not, and abstain with an escalation if still weak. Never veto on a guess.

4 State and Message Schemas

```
plan_state = {
  "run_id": str, "version": int, "round": int,
  "disruption": {"type": "fog"|"aog", "airport": str, "start": ts, "end": ts},
  "flights":   [{"flight_id","tail","origin","dest","std","sta","status"}],
  "crew":      [{"crew_id","role","duty_start","flight_ids"}],
  "pax":       {"affected_bookings": [...], "missed_connections": int},
  "constraints_log": [{"agent","clause_id","reason","round"}],
  "candidates": [{"plan_id","plan_diff","scores":{...},"vetoed_by":[...]}]
```

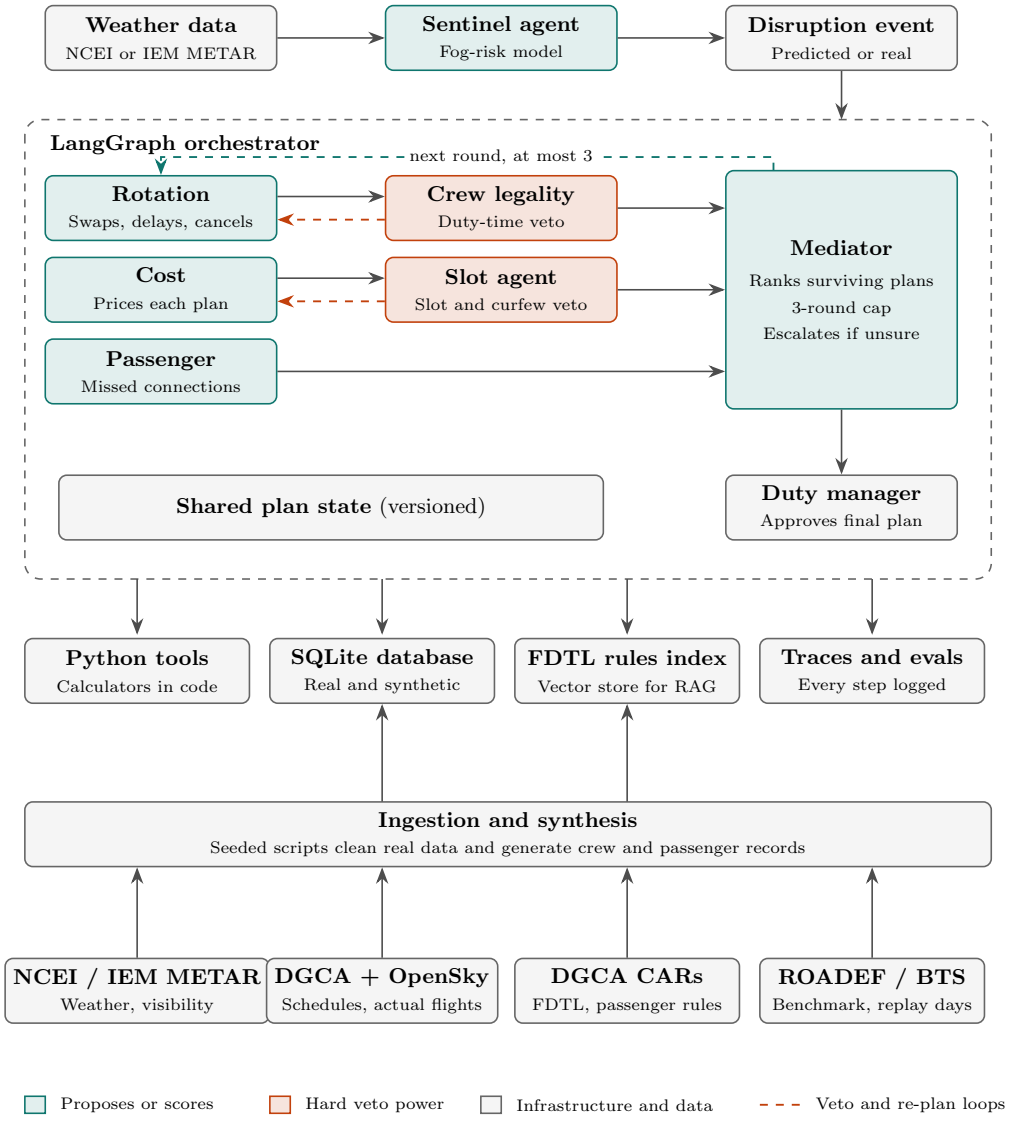


Figure 1: Architecture, loops and data. Real sources (bottom) are cleaned by seeded ingestion scripts, which also generate the synthetic crew and passenger records, and fill SQLite and the rules index. A veto returns to the proposers as a counter (orange dashed). The Mediator can send the plan back for another round, at most three (teal dashed), then recommends a plan or escalates, and the duty manager approves. ROADEF and BTS also serve as evaluation data.

```
}
message = {
  "from": str, "to": str,
  "type": "PROPOSE"|"VETO"|"COUNTER"|"ACCEPT"|"ABSTAIN",
  "plan_diff": [{"op":"swap_tail|delay|cancel|reposition_crew", ...}],
  "rationale": str, "evidence_refs": [str]   # tool result ids, clause ids
}
trace_span = {"run_id","round","agent","tool","input","output","ts","tokens"}
```

5 Agentic Patterns and Rationale

Tool use. All duty-time, missed-connection and cost figures come from code, so they are checkable and the model never does arithmetic. **Agentic RAG.** The crew agent controls its own retrieval, as above. **Planning.** The loop is propose, check, veto, re-plan, with a reflection step in the Mediator. **Multi-agent.** The goals genuinely conflict and each agent has its own tools and data. Hard veto agents sit next to soft agents, so legality can never be traded for a better score. **Human-in-the-loop.** The final plan goes to the duty manager through a LangGraph interrupt. **Observability.** Every prediction, proposal, veto, tool call and score is a trace span. **Experimentation.** Single-agent vs multi-agent, reactive vs predictive, and ablations (Section 9).

Agent	Role and output	Tools	Power
Sentinel	Predicts a disruption early. Output: {event, probability, expected start, lead time, confidence}, then monitor, pre-position or act early	`get_forecast`, `fog_risk_model`, `aog_risk`	Advises only
Rotation	Proposes aircraft swaps, delays and cancellations as plan diffs	`query_schedule`, `swap_tail`, `check_range_maint`	Proposes
Crew legality	Checks duty time and rest against the FDTL CAR. Output: PASS or VETO with cited clause, or abstain	`calc_duty_time`, `retrieve_fdtl`	Veto
Slot	Checks airport slots and curfew. Output: PASS or VETO	`check_slot`	Veto
Passenger	Counts missed connections, finds rebooking options	`get_affected_bookings`, `calc_missed_connections`, `get_rebooking_options`	Scores
Cost	Prices each plan (delay, cancellation, compensation, meals, hotels)	`estimate_cost`	Scores, may propose cheaper options
Mediator	Runs rounds, hard-filters vetoed plans, scores, reflects, escalates	`score_plan`, round counter, escalation	Decides

Table 1: Agents, outputs, tools and decision power.

LangGraph is chosen over CrewAI or AutoGen because the design is a loop with shared state, conditional exits and an approval pause.

6 Data Plan

6.1 Sources

Rules. The crew rules come from DGCA CAR Section 7, Series J, Part III (flight and duty time limitations and rest). Revised norms were reported with 48-hour weekly rest and limits on night operations, and there were postponements, so **confirm the version in force** before coding. The passenger-facility rules are DGCA CAR Section 3, Series M, Part IV (cancellation, delay, denied boarding). Reported points: meals for longer delays, hotel facilities for long or overnight delays, an offer of alternate flight or refund on cancellation, and no compensation for extraordinary circumstances. Read the CAR for the exact thresholds and amounts.

6.2 Hybrid data design

Pick one hub airport (an Indian airport with winter fog is the intended story; **UNVERIFIED** that schedule, METAR and OpenSky data are all obtainable for it). Real flights come from the DGCA schedule, or from BTS as the fallback. Real weather comes from IEM or NCEI. Actual movements come from OpenSky or BTS. Synthetic, seeded generators then produce crew pairings and passenger bookings and connections anchored to those real flights. Do a one-hour feasibility check first (Section 11, step 0), and switch to BTS for flights if any Indian piece fails.

6.3 Database tables (SQLite)

Plant known answers and a few deliberately messy cases (missing connection time, no seats, ambiguous rule) so abstention can be tested.

7 Tool Contracts

8 Stack and Constraints

LangGraph for orchestration (state, conditional edges for the round cap, interrupt for approval); Python functions for tools; SQLite; a local vector store for the FDTL index; scikit-learn for the fog model; Langfuse or JSONL traces; a free-tier or local language model. External services are mocked. Every loop is bounded (three rounds, token budget, improvement threshold). The system is advisory and never presented as an operational decision. **To decide:** the exact model, the embedding model, and whether the demo is a command line tool or a small web view of the traces (default: command line plus a trace viewer).

Source	What it provides	Used for	Status and caveat
ROADEF 2009 Challenge (Amadeus)	Public instances from a European airline, with aircraft rotations, passenger itineraries and an evaluation function that weights passenger recovery. Published solver results exist for comparison	Benchmark for recovery quality; passenger side	Instances confirmed. **UNVERIFIED:** licence terms and the exact evaluation function; read the challenge PDF
BTS On-Time Performance (US)	Flight-level data, public, monthly, with tail numbers, cancellation reasons and delay-cause categories. Delay-cause minutes recorded only for delays of 15 min or more	Replay of real disruption days; rotation reconstruction	Confirmed. US data, so mixing with DGCA rules is a hybrid. The transtats link may be down
IEM ASOS/METAR archive	Airport weather observations worldwide, including visibility	Fog-risk model labels and features	Confirmed to exist. **UNVERIFIED:** coverage of the chosen Indian airport
NOAA NCEI	Historical weather archives	Alternative weather source	**UNVERIFIED:** not opened; check airport coverage
OpenSky scientific datasets	Monday state vectors (2017–2022) at 10 s intervals, aircraft metadata, a one-day full-table snapshot from March 2026	Actual movements and delays; aircraft type	Confirmed. No schedules, cancellations, crew or passengers; Mondays only for the weekly files
DGCA portal	Flight schedule pages, CARs, circulars, monthly and annual domestic traffic data, incident reports	Schedules, rules, cost rules	Pages confirmed. **UNVERIFIED:** schedule file format and whether flight-level delays exist (traffic data appears aggregate)
data.gov.in	National open data portal	Possible traffic baselines	**UNVERIFIED:** not opened

Table 2: Real data sources and their status.

9 Evaluation Plan and Metrics

Benchmarks and baselines. Run the system on ROADEF instances and compare with the published solver results. Replay real disruption days from BTS or OpenSky and compare with what actually happened. Baselines: a greedy rule-based recovery, and an optimisation solver such as OR-Tools. Test set: 25 or more scenarios with known answers, at least 5 adversarial (missing data, ambiguous rule).

Experiments. Single-agent vs multi-agent, reactive vs predictive, one vs three negotiation rounds, and the ablations above, all on the same scenarios.

Statistics. Run each scenario several times because model output varies. Report means with confidence intervals from a paired bootstrap, use a Wilcoxon signed-rank test, and report effect sizes.

How to frame the claim. An optimisation solver will probably win on raw cost. The honest and publishable claim is that agents retrieve and apply rules correctly, explain rejections, and abstain on messy cases while staying close to the solver on cost. Report negative results plainly.

Limitations to state. Crew and passenger data are synthetic. Mixing US or European flight data with Indian rules is a hybrid. Weather labels from real data still do not prove forecasting skill beyond the tested airport and period. Cost rates are assumptions unless taken from a source.

10 Repository Layout

```
data/raw/  data/processed/          # real downloads, built database and index
ingest/    fetch_weather.py fetch_schedule.py load_opensky.py
           build_rules_index.py synth_crew_pax.py make_scenarios.py
tools/     schedule.py duty.py slots.py passengers.py cost.py scoring.py
agents/    sentinel.py rotation.py crew.py slot.py passenger.py cost.py mediator.py
graph/     state.py build_graph.py
eval/      validator_fdtl.py metrics.py baselines/ run_eval.py rag_gold.jsonl
traces/    # JSONL spans, one file per run
app/       cli.py trace_viewer.py
```

Table	Key fields	Origin
flights	flight_id, tail, origin, dest, std, sta, seats, status	real
aircraft	tail, type, seats, min_turn_min	real or synthetic
weather_obs	station, ts, visibility, wind, metar_raw	real
crew_pairings	crew_id, role, base, duty_start, flight_ids	synthetic
bookings	booking_id, pax_count, fare_class, avg_fare	synthetic
itinerary_legs	booking_id, leg_no, flight_id	synthetic
connections	booking_id, inbound_flight, outbound_flight, min_connect_min	synthetic
slots	airport, window_start, window_end, capacity, curfew_start, curfew_end	synthetic or real
cost_rates	item, rate	assumed, documented
scenarios	scenario_id, type, start, end, label, difficulty	generated

Table 3: Tables, with real or synthetic origin. Everything synthetic uses a fixed random seed.

Tool	Input	Output
get_forecast	airport, window	feature rows from weather_obs
fog_risk_model	features	probability, confidence
aog_risk	tail	risk score from fault trend
query_schedule	flight_ids or window	flight rows
swap_tail	plan, flight_id, new_tail	ok flag, reasons
check_range_maint	tail, flight_id	ok flag, reasons
calc_duty_time	crew_id, plan	duty and flight-duty minutes, violations
retrieve_fdtl	query, k	chunks with clause ids and scores
check_slot	plan_diff	ok flag, violated windows
get_affected_bookings	flight_ids	booking rows
calc_missed_connections	plan_diff	count and list
get_rebooking_options	booking_id, plan	feasible flights with seats
estimate_cost	plan_diff	delay, cancellation, compensation, meals, hotel, total
score_plan	plan, weights	scalar score and components
validate_plan_independent	plan	violations. **Evaluation only; agents must not call it**

Table 4: Tool signatures. All deterministic except `retrieve_fdtl` and `fog_risk_model`.

```
tests/     # unit tests for every tool
README.md  # run instructions
```

11 Build Order, Owners and Dates

Owners: Athish Raj, data and tools; Dinesh K, agents and orchestration; Sanchit Rajesh, tracing, evaluation and experiment. Everyone must be able to explain the whole system. Dates: Review 3 on 31 October (live core loop), Review 4 on 7 November (full system, results, report), final submission by 12 November.

Step 0. Data feasibility check (1 hour). For one chosen airport, confirm a usable schedule, METAR history, and OpenSky or BTS actuals. Output: a short note and the fallback choice. *Done when:* the data source decision is written down.

Step 1. Schema, ingestion and synthetic generator. Build the tables, load real data, generate crew and passengers with a fixed seed, and create five scenarios. *Done when:* one command rebuilds the database identically.

Step 2. Deterministic tools with unit tests. Duty time, missed connections, cost, slot, scoring. *Done when:* duty-time results match hand-worked examples from the CAR.

Step 3. Independent FDTL validator in `eval/`, separate code path from the agent tool.

Step 4. Graph skeleton. LangGraph with stub agents, the Mediator loop, the round cap, and JSONL traces. *Done when:* a scripted run produces a trace with rounds and messages.

Step 5. Real Rotation, Crew (with RAG) and Slot agents, one veto loop. *Done when:* the fog demo shows a proposal, a cited veto, a counter, and a recommendation. This is the Review 3 target.

Step 6. Passenger and Cost agents, scoring, escalation, human approval interrupt.

Step 7. Baselines and evaluation harness. Greedy and solver baselines, metrics, ROADEF adapter, RAG

Group	Metric	How it is measured
Recovery quality	Disrupted passengers, passenger delay, cancellations, cost	On ROADEF instances against published results; on replay days against actual outcomes, greedy and solver
Rule compliance	FDTL violation rate	By a separate validator coded directly from the CAR, not by the agent's own tool, so the result is not circular
Prediction	Precision, recall, lead time, Brier score	Time-based train and test split on real weather; also net cost saved by pre-positioning minus false-alarm cost
Abstention	Risk-coverage curve	Accuracy as a function of how often the agent answers
Retrieval quality	Correct-clause recall, citation precision	Gold set of about 50 FDTL questions with annotated clauses
Efficiency	Rounds, tokens, latency, cost per run	From traces
Ablations	Effect of each part	Remove RAG, remove reflection, remove one agent at a time
Human judgment	Rubric rating, agreement	Two or three domain experts rate plans; report inter-rater agreement; validate any LLM judge against them

Table 5: Metrics.

gold set.

Step 8. Sentinel and fog model on real weather with a time-based split.

Step 9. Experiments, ablations, statistics, report.

If time runs short, drop the Sentinel first, then the Slot agent. The core loop must work before anything else is added.

12 Open Questions

Which Indian airport and winter period. Whether DGCA schedule files are machine readable. Coverage of IEM or NCEI for that airport. ROADEF licence and the exact evaluation function. The FDTL version in force and exact passenger-rule thresholds. Which language model and embedding model to use. Command line or web view for the demo. Who is team lead.

References and Links

- ROADEF 2009 Challenge, Disruption Management for Commercial Aviation: `https://roadef.org/challenge/2009/en/` and `https://www.roadef.org/challenge/2009/files/challenge_en.pdf`
- Artigues et al., Disruption management for commercial airlines: methods and results for the ROADEF 2009 Challenge, EJIE 6(6), 2012: `https://hal.archives-ouvertes.fr/hal-00492035`
- BTS airline on-time performance and causes of delay: `https://www.bts.gov/content/major-us-air-carrier-delays-cancellations-and-diversions`
- Iowa Environmental Mesonet ASOS/METAR archive: `https://mesonet.agron.iastate.edu/request/download.phtml`
- NOAA NCEI: `https://www.ncei.noaa.gov`
- OpenSky scientific datasets: `https://opensky-network.org/data/scientific`
- DGCA portal (CARs, schedules, statistics): `https://www.dgca.gov.in/digigov-portal`
- DGCA CAR Section 7, Series J, Part III (FDTL) and CAR Section 3, Series M, Part IV (passenger facilities), from the DGCA portal
- Open Government Data portal: `https://www.data.gov.in/`